

EXAMEN FINAL

Materia: Ingeniería de Software II

Modalidad: Individual, con uso de computadora

Distribución: 90% práctica y 10% teoría

Recursos permitidos: IDE, navegador, Git/GitHub, documentación básica de herramientas, editor de texto.

Enunciado. -

La organización para la cual trabajas necesita una aplicación básica de gestión de tareas para organizar actividades internas del equipo. El sistema actualmente presenta un código desordenado, poca claridad en la estructura, ausencia de pruebas y un uso deficiente del control de versiones.

Se te entrega una estructura base del proyecto. Tu tarea consiste en desarrollar la estructura, mejorar, organizar y validar técnicamente el sistema aplicando buenas prácticas de ingeniería de software.

Debes trabajar de manera individual, usando tu computadora, y demostrar que puedes desarrollar software con enfoque profesional.

Estructura del examen. -

A. Parte teórica: 10%

Puntaje: 10 puntos

Responde de forma breve y precisa:

1. ¿Cuál es la diferencia entre arquitectura monolítica y arquitectura por capas?
2. ¿Por qué la refactorización no debe cambiar la funcionalidad del software?
3. ¿Cuál es la importancia de las pruebas automatizadas en un proyecto de software?

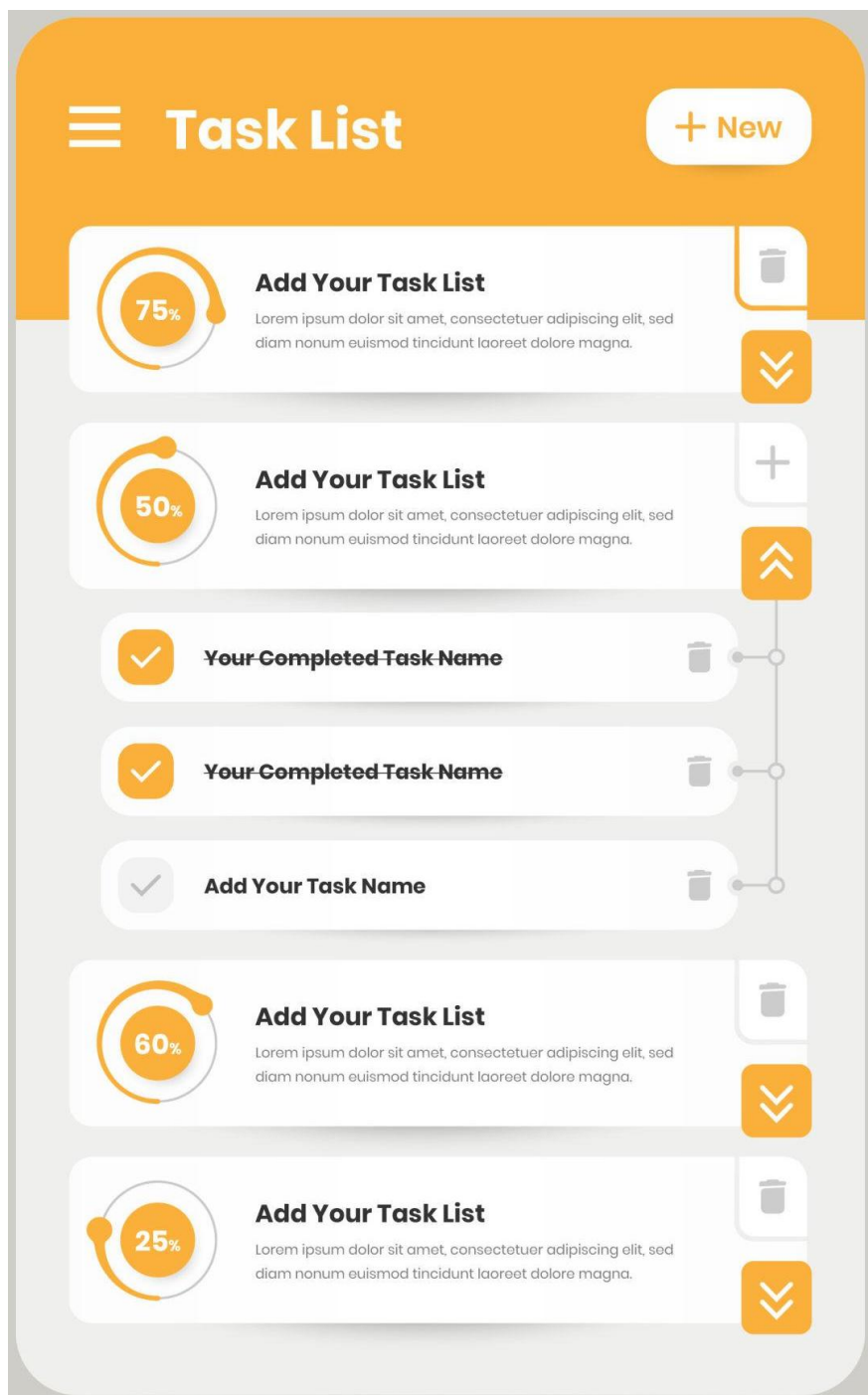
B. Parte práctica: 90%

Puntaje: 90 puntos

Estructura genérica, para desarrollar la estructura base del proyector:

- Backend: Java (Spring), Node.js, Python, .NET, etc.
- Frontend: React, Angular, Vue, HTML/JS
- BD: MySQL, PostgreSQL, MongoDB, SQLite

Ejemplo de frontend



Ejemplo de la Estructura

project-exam/

|

└─ backend/

|

└─ src/

|

└─ config/

|

└─ controllers/

|

└─ services/

|

└─ models/

|

└─ repositories/

|

└─ routes/ (opcional)

|

|

└─ tests/

|

└─ main.(js|py|java|cs)

|

└─ frontend/

|

└─ src/

|

└─ components/

|

└─ pages/

|

└─ services/

|

└─ assets/

|

|

└─ index.(html|js|jsx|ts)

|

└─ database/

|

└─ schema.sql (o esquema NoSQL)

|

└─ seed.sql (datos iniciales opcional)

|

└─ docs/

```
| └─ architecture.md
|
| └─ tests/
|   └─ integration/
|
| └─ .env
| └─ .gitignore
└─ README.md
```

Actividad 1. Organización inicial del proyecto y control de versiones

Debes realizar lo siguiente:

1. Desarrollar la estructura base del proyecto en base a la imagen dada por el docente
2. Clonar o abrir el proyecto base.
3. Crear una rama de trabajo con nombre: examen-nombre-apellido.
4. Verificar el estado inicial del repositorio.
5. Realizar al menos 3 commits con mensajes claros y técnicos.
6. Subir o dejar preparado el trabajo en la rama correspondiente.

Actividad 2. Identificación de estructura y arquitectura del sistema

Debes revisar el proyecto y redactar en el archivo README.md o en un comentario técnico breve:

1. Qué tipo de arquitectura tiene el proyecto base.
2. Qué módulos o componentes identifica.
3. Qué mejoras arquitectónicas propondrías para hacerlo más mantenible.

Actividad 3. Refactorización de código

Realiza al menos tres mejoras, por ejemplo:

- renombrar variables o funciones con nombres más claros
- eliminar código duplicado
- dividir una función larga en funciones más pequeñas
- mejorar legibilidad

- organizar mejor la estructura del archivo o módulo

Actividad 4. Implementación de una mejora funcional pequeña

Debes agregar una funcionalidad sencilla al sistema, por ejemplo:

- registrar una nueva tarea
- listar tareas pendientes
- marcar una tarea como completada
- validar que no se ingresen datos vacíos

Actividad 5. Pruebas de software

Debes crear al menos 2 pruebas automatizadas para validar una parte del sistema.

Las pruebas pueden verificar:

- suma, validación o procesamiento simple
- una función de registro
- una regla de negocio
- una salida esperada del sistema

Además, debes ejecutar las pruebas y confirmar que pasan correctamente.

Entregables obligatorios

Al finalizar el examen debes presentar:

1. El código organizado.
2. La rama de trabajo con commits realizados.
3. El archivo README.md o nota técnica breve con el análisis arquitectónico.
4. Las pruebas automatizadas creadas.
5. Evidencia de ejecución correcta de pruebas.

Indicaciones a tomar en cuenta

- Trabaja de forma individual.
- Guarda tus cambios constantemente.
- Usa nombres claros en archivos, funciones y commits.
- No copies código sin comprenderlo.
- Prioriza que el sistema funcione y que tu solución esté bien documentada.
- Si no terminas todo, se valorará lo que quede bien implementado y justificado.